Nama: Muhammad Fadhil Ilyas
NIM: 434241099
Kelas: TI-C6
Link Github: https://github.com/ilysthecreator/PBL_Praktikum.git

Modul 11: Background Jobs & Asynchronous Processing (Edisi Puzzle — Analisis dan Perbaikan Kesalahan Sistem Asinkron)

D.1 Berita Acara Perbaikan (Tabel 9 Kesalahan)

Modul 11 Edisi Puzzle memuat 9 kesalahan yang sengaja ditanam pada kode sumber awal. Berdasarkan metodologi 'Berburu Kesalahan pada Sistem Asinkron' (Bagian B.0), kesalahan dianalisis secara bertahap mulai dari tingkat compiler (3 kesalahan kompilasi) hingga tingkat pengujian logika konkurensi runtime (6 kesalahan logika sistem asinkron). Seluruh kesalahan telah ditemukan, dibuktikan gejalanya, dan diperbaiki hingga seluruh spesifikasi penerimaan (Bagian C) terpenuhi secara sempurna.

No	Lokasi (File, Fungsi)	Gejala yang Diamati	Uji yang Mengungkap	Akar Masalah	Perbaikan
1	helper/mailer.go (NewLogMailer)	Compiler error: tipe LogMailer tidak mengimplementasikan interface Mailer.	Compiler (go build ./helper/...)	Fungsi NewLogMailer mengembalikan nilai konkret (value) LogMailer{...}, sedangkan method Send dideklarasikan dengan pointer receiver (*LogMailer). Selain gagal kompilasi, pengembalian value menyalin mutex dan atomic counter.	Mengembalikan pointer &LogMailer{...} agar memenuhi interface Mailer dan mutex/counter dibagi bersama seluruh goroutine worker.
2	app/service/ course_service.go (Swap)	Compiler error: undefined variable 'to' saat membentuk struct event outbox.	Compiler (go build ./app/...)	Pada method Swap, hasil panggilan IncrementEnrolled disimpan pada variabel 'joined', namun pemanggilan s.outbox.WithTx(tx).Insert merujuk ke variabel 'to' yang tidak dideklarasikan.	Mengganti to.ID, to.Code, dan to.Name menjadi joined.ID, joined.Code, dan joined.Name.
3	app/service/ notification_worker.go (HandleSend)	Compiler error: method HandleSend tidak cocok dengan signature asynq.HandlerFunc.	Compiler (go build ./app/...)	Parameter task pada HandleSend dideklarasikan sebagai value t asynq.Task, sedangkan Asynq mewajibkan pointer t *asynq.Task pada mux.HandleFunc.	Mengubah signature parameter menjadi func (w *NotificationWorker) HandleSend(ctx context.Context, t *asynq.Task) error.
4	app/service/ course_service.go (Enroll)	Mahasiswa menerima email 'berhasil terdaftar' padahal pendaftaran gagal (409 ALREADY_ENROLLED).	Uji C.1 (Permintaan 2)	Pencatatan event ke outbox memanggil s.outbox.Insert(ctx, ...) langsung ke connection pool tanpa s.outbox.WithTx(tx). Saat langkah 5 (InsertEnrollment) gagal dan tx rollback, outbox tidak ikut batal.	Mengubah pemanggilan menjadi s.outbox.WithTx(tx).Insert(...) agar pencatatan outbox terikat pada transaksi database yang sama.
5	app/service/	Mahasiswa yang	Uji C.1	Konstanta tipe event	Mengganti tipe event

	course_service.go (Drop)	membatalkan kelas (Drop) menerima email konfirmasi 'KRS: berhasil terdaftar'.	(Permintaan 3 & verifikasi tabel)	yang dipassing ke outbox adalah model.EventEnrollmentCreated, bukan EventEnrollmentDropped.	menjadi model.EventEnrollmentDropped sehingga subjek email yang disusun adalah 'KRS: keluar dari <kelas>'.
6	app/service/ outbox_relay.go (RelayOnce)	Ketika Redis mati, event yang gagal di-enqueue ditandai terbit (published_at terisi). Notifikasi hilang permanen saat Redis menyala kembali.	Uji C.2 (Redis mati, KRS tetap jalan)	Baris ids = append(ids, e.ID) dieksekusi SEBELUM memanggil r.publish(ctx, e). Saat publish gagal, ID sudah masuk ke slice ids dan ditandai oleh MarkPublished.	Memindahkan ids = append(ids, e.ID) setelah blok pengecekan error r.publish berhasil, sehingga hanya event yang benar-benar masuk Redis yang ditandai.
7	config/queue.go (NewWorkerServer)	Worker berjalan tanpa error tetapi tidak pernah memproses email. Task menumpuk di pending selamanya.	Langkah 11 / Uji C.1	Konfigurasi Queues pada asynq.Config diisi map[string]int{model.TaskSendNotification: 1}. Worker mendengarkan nama task type, bukan nama antrean (QueueNotifications).	Mengganti nama antrean yang didengarkan worker menjadi Queues: map[string]int{model.QueueNotifications: 1}.
8	app/service/ notification_worker.go (HandleSend)	Pada kegagalan sementara (temporary failure), email tidak pernah di-retry dan hilang.	Uji C.3 (Temporary failure)	Panggilan w.notifications.Claim dijalankan di luar transaksi (tanpa WithTx(tx)). Saat mailer gagal dan tx rollback, baris claim tetap tersimpan di database. Pada retry berikutnya, claim mengembalikan false sehingga pengiriman email dilewati.	Mengubah menjadi w.notifications.WithTx(tx).Claim(...) di dalam WithTxOnce sehingga jika mailer gagal, baris claim ikut dibatalkan.
9	app/service/ notification_worker.go (HandleSend)	Alamat email tidak valid (.invalid) di-retry 5 kali hingga attempt 6/6, tidak langsung masuk archived.	Uji C.4 (Permanent failure)	Error permanent failure dibungkus dengan fmt.Errorf("%v: %w", asynq.SkipRetry, err). Penggunaan verb %v mengubah SkipRetry menjadi string polos sehingga errors.Is bernilai false.	Mengubah format pembungkusan error menjadi fmt.Errorf("%w: %v", asynq.SkipRetry, err) dengan verb %w untuk asynq.SkipRetry.

Rincian Bukti Sebelum dan Sesudah Perbaikan

1. Bukti Kesalahan 1, 2, dan 3 (Tingkat Compiler):

```
// Kompilasi awal gagal dengan 3 error bertahap:
# Error 1 (helper/mailer.go):
./helper/mailer.go:50:9: cannot use LogMailer{...} (value of type LogMailer) as Mailer value in
return statement:
    LogMailer does not implement Mailer (method Send has pointer receiver)

# Error 2 (app/service/course_service.go):
./app/service/course_service.go:210:32: undefined: to

# Error 3 (app/service/notification_worker.go):
./app/service/notification_worker.go:45:16: cannot use worker.HandleSend (value of type func(ctx
context.Context, t asynq.Task) error) as asynq.HandlerFunc value in argument to mux.HandleFunc

// Setelah ketiga berkas diperbaiki, perintah kompilasi dan go vet berjalan bersih tanpa
peringatan:
```

```
PS C:\Kuliah\Backend\Modul-11> go build ./app/... ./helper/...
PS C:\Kuliah\Backend\Modul-11> go vet ./app/... ./helper/...
(Exit code 0, tidak ada error kompilasi)
```

2. Bukti Kesalahan 4 (Enroll Tanpa WithTx):

```
// SEBELUM PERBAIKAN: POST /courses/2/enroll dijalankan dua kali.
// Panggilan kedua menghasilkan 409 ALREADY_ENROLLED, namun tabel outbox tetap mencatat event!
SELECT id, event_type, (payload->>'user_id')::int AS uid FROM outbox ORDER BY id DESC LIMIT 2;
 id |   event_type       | uid
----+--------------------+-----
 15 | enrollment.created | 42   <-- Event haram dari request yang gagal 409!
 14 | enrollment.created | 42   <-- Event asli yang berhasil 201

// SESUDAH PERBAIKAN: s.outbox.WithTx(tx).Insert(...)
// Saat request kedua gagal 409, transaksi PostgreSQL rollback dan tabel outbox bersih:
SELECT id, event_type, (payload->>'user_id')::int AS uid FROM outbox ORDER BY id DESC LIMIT 2;
 id |   event_type       | uid
----+--------------------+-----
 14 | enrollment.created | 42   <-- TEPAT 1 event, request gagal tidak meninggalkan jejak!
```

3. Bukti Kesalahan 5 (Tipe Event Drop Keliru):

```
// SEBELUM PERBAIKAN: Mahasiswa melakukan Drop kelas, subjek email yang dikirim salah:
// Log mail.log:
2026-11-08T20:10:05+07:00 to=mhsuji@student.unair.ac.id subject="KRS: berhasil terdaftar di IF102
Basis Data Lanjut"

// SESUDAH PERBAIKAN: Menggunakan model.EventEnrollmentDropped:
// Log mail.log:
2026-11-08T20:12:10+07:00 to=mhsuji@student.unair.ac.id subject="KRS: keluar dari IF102 Basis
Data Lanjut"
```

4. Bukti Kesalahan 6 (Outbox Relay Append ids Prematur):

```
// SEBELUM PERBAIKAN: Redis dimatikan, 3 mahasiswa mendaftar KRS.
// Relay menandai published_at terisi karena ID dimasukkan ke list sebelum publish gagal!
SELECT id, published_at IS NOT NULL AS di_publish FROM outbox ORDER BY id DESC LIMIT 3;
 id | di_publish
----+------------
 20 | true   <-- SALAH! Redis mati tetapi ditandai sudah di-publish!
 19 | true
 18 | true
// Akibatnya saat Redis dinyalakan kembali, 0 email terkirim (notifikasi hilang permanen).

// SESUDAH PERBAIKAN: ids = append(ids, e.ID) dipindahkan setelah err == nil.
SELECT id, published_at IS NOT NULL AS di_publish FROM outbox ORDER BY id DESC LIMIT 3;
 id | di_publish
----+------------
 20 | false  <-- BENAR! Tetap NULL selama Redis belum berhasil menerima.
 19 | false
 18 | false
// Saat Redis dinyalakan, ketiga event otomatis di-publish dan terkirim tuntas.
```

5. Bukti Kesalahan 7 (Nama Queue Salah di config/queue.go):

```
// SEBELUM PERBAIKAN: Worker mendengarkan queue 'notification:send' (Task Type):
GET /admin/queue
{ "outbox_pending": 0, "outbox_published": 4, "queue": { "pending": 4, "active": 0,
"processed_today": 0 } }
// Seluruh task menunggu di queue 'notifications' selamanya tanpa disentuh worker.
```

```
// SESUDAH PERBAIKAN: Worker mendengarkan queue 'notifications':
GET /admin/queue
{ "outbox_pending": 0, "outbox_published": 4, "queue": { "pending": 0, "active": 0,
"processed_today": 4 } }
// Seluruh task langsung diproses dan berpindah ke processed_today.
```

6. Bukti Kesalahan 8 (Claim Notifikasi di Luar Transaksi):

```
// SEBELUM PERBAIKAN: MAILER_FAIL_EVERY=3 (uji C.3).
// Saat attempt 1 gagal sementara, claim tetap tersimpan di database. Saat Asynq melakukan retry,
// Claim melihat baris sudah ada dan mengabaikan pengiriman email (alreadySent = true).
// Hasil mail.log: Hanya 4 email terkirim (2 email gagal tidak pernah dikirim ulang!).

// SESUDAH PERBAIKAN: w.notifications.WithTx(tx).Claim(...)
// Saat attempt 1 gagal, claim rollback. Retry berhasil melakukan claim dan mengirim ulang:
// Hasil mail.log: Tepat 6 baris email terkirim, sesuai dengan 6 baris di notifications.
```

7. Bukti Kesalahan 9 (SkipRetry Menggunakan Verb %v Bukan %w):

```
// SEBELUM PERBAIKAN: Kirim email ke salah@kampus.invalid (uji C.4).
// Log worker menunjukkan 5 kali retry karena SkipRetry tidak terdeteksi:
{"level":"WARN","msg":"task_failed","type":"notification:send","attempt":"1/6","error":"skip
retry for the task: alamat salah@kampus.invalid tidak ditemukan"}
{"level":"WARN","msg":"task_failed","type":"notification:send","attempt":"2/6","error":"..."}
...
{"level":"WARN","msg":"task_failed","type":"notification:send","attempt":"6/6","error":"..."}

// SESUDAH PERBAIKAN: Menggunakan %w: fmt.Errorf("%w: %v", asynq.SkipRetry, err)
// Log worker TEPAT SATU KALI attempt dan langsung masuk archived:
{"level":"WARN","msg":"task_failed","type":"notification:send","attempt":"1/6","error":"skip
retry for the task: alamat salah@kampus.invalid tidak ditemukan"}
// Response GET /admin/queue:
{ "queue": { "archived": 1, "retry": 0 } }
```

D.2 Notifikasi Perubahan Kuota untuk Seluruh Peserta (Tugas Mandiri)

1. Alasan UpdateQuota Memaksa Penggunaan Transaksi (WithTx)

Pada Modul 9, fungsi UpdateQuota hanya mengeksekusi satu pernyataan SQL tunggal: UPDATE courses SET quota = $1, version = version + 1 WHERE id = $2 AND version = $3. Di PostgreSQL, setiap pernyataan SQL tunggal secara otomatis dijamin atomik oleh database engine tanpa perlu deklarasi transaksi eksplisit. Namun, ketika fitur notifikasi perubahan kuota ditambahkan, sistem wajib melakukan dua operasi sekaligus: (1) memperbarui kuota kelas di tabel courses, dan (2) mencatat event baru course.quota_changed ke dalam tabel outbox. Jika kedua operasi ini tidak dibungkus dalam transaksi yang sama (WithTx), sistem akan rentan terhadap anomali Dual Write: apabila pembaruan kuota berhasil tetapi aplikasi crash atau koneksi database terputus sebelum pencatatan outbox selesai, kuota kelas berubah tanpa pernah ada notifikasi yang dikirim ke mahasiswa. Sebaliknya, bila pencatatan outbox dilakukan lebih dahulu dan pembaruan kuota gagal (misalnya karena konflik versi optimistic locking), mahasiswa akan menerima email palsu bahwa kuota telah berubah. Oleh karena itu, penggunaan transaksi eksplisit (s.tx.WithTx) mutlak diwajibkan agar pembaruan data dan pencatatan event outbox bersifat atomik (all-or-nothing).

2. Skrip Migrasi Database (migrations/009_quota_notifications.sql)

Pada skema awal (migrations/008_outbox.sql), tabel notifications memiliki constraint UNIQUE (outbox_id). Constraint ini dirancang dengan asumsi relasi 1 event outbox menghasilkan tepat 1 notifikasi

(seperti saat pendaftaran individu mahasiswa). Pada fitur perubahan kuota kelas, 1 event outbox_id harus disiarkan (broadcast) ke seluruh peserta kelas yang terdaftar (1 event menghasilkan N notifikasi). Bila UNIQUE (outbox_id) dipertahankan, mahasiswa kedua dan seterusnya akan ditolak oleh database akibat pelanggaran constraint unik. Oleh karena itu, skrip migrasi 009 menghapus constraint lama dan menggantinya dengan komposit unik UNIQUE (outbox_id, user_id):

```
-- migrations/009_quota_notifications.sql
-- Mengizinkan satu event outbox menghasilkan banyak notifikasi (broadcast ke banyak user),
-- namun tetap menjamin idempotensi: satu user hanya menerima satu notifikasi per event.

ALTER TABLE notifications
    DROP CONSTRAINT IF EXISTS notifications_outbox_unique;

ALTER TABLE notifications
    ADD CONSTRAINT notifications_outbox_user_unique UNIQUE (outbox_id, user_id);
```

3. Desain Worker dan Penanganan Partial Failure

Terdapat dua pendekatan dalam merancang worker untuk pengiriman massal: (A) Relay memecah event menjadi N task di Redis (satu task per mahasiswa), atau (B) Satu task dikirim ke Redis, dan worker melakukan perulangan pengiriman ke seluruh peserta di dalamnya. Pendekatan yang dipilih adalah pendekatan satu task dengan pemrosesan atomik per peserta di dalam worker. Kunci ketahanan sistem terhadap partial failure (misalnya email ke-7 dari 40 email gagal akibat server email sibuk) adalah penggunaan transaksi claim per mahasiswa menggunakan constraint UNIQUE (outbox_id, user_id): Setiap iterasi mahasiswa membungkus pencatatan claim dan pengiriman email dalam blok transaksi mandiri. Ketika pengiriman email ke-7 gagal, transaksi mahasiswa ke-7 dibatalkan (rollback claim) dan worker mengembalikan error ke Asynq untuk memicu mekanisme retry. Ketika Asynq menjalankan retry beberapa saat kemudian, worker membaca kembali daftar 40 peserta. Mahasiswa ke-1 sampai ke-6 yang sudah sukses pada attempt sebelumnya telah memiliki baris di tabel notifications; pemanggilan Claim() untuk mereka mengembalikan false (ON CONFLICT DO NOTHING), sehingga worker langsung melewati pengiriman email untuk mahasiswa 1–6 (alreadySent = true) dan langsung memproses pengiriman mulai dari mahasiswa ke-7. Dengan demikian, retry tidak pernah mengirimkan email ganda kepada mahasiswa yang sudah berhasil menerima.

4. Potongan Kode Penting D.2 (CourseService & Worker)

```
// 1. app/service/course_service.go (UpdateQuota dengan Outbox Event)
func (s *CourseService) UpdateQuota(ctx context.Context, id int, req model.UpdateQuotaRequest)
(model.Course, error) {
    var updated model.Course
    err := s.tx.WithTx(ctx, func(tx repository.DBTX) error {
        repo := s.courses.WithTx(tx)
        outboxRepo := s.outbox.WithTx(tx)

        var err error
        updated, err = repo.UpdateQuota(ctx, id, req.Quota, req.Version)
        if err != nil {
            return err
        }

        // Catat event perubahan kuota ke outbox dalam transaksi yang sama
        eventPayload := model.QuotaChangedEvent{
            CourseID:   updated.ID,
            CourseCode: updated.Code,
            CourseName: updated.Name,
            NewQuota:   updated.Quota,
        }
```

```
        return outboxRepo.Insert(ctx, "course.quota_changed", eventPayload)
    })
    return updated, err
}

// 2. app/service/notification_worker.go (Penanganan Event course.quota_changed)
func (w *NotificationWorker) handleQuotaChanged(ctx context.Context, job model.NotificationJob)
error {
    var ev model.QuotaChangedEvent
    _ = json.Unmarshal(job.Payload, &ev)

    // Ambil seluruh peserta aktif kelas dari database
    students, _ := w.courses.GetEnrolledStudents(ctx, ev.CourseID)

    for _, student := range students {
        msg := model.Message{
            To:      student.Email,
            Subject: fmt.Sprintf("KRS: kuota %s %s berubah menjadi %d", ev.CourseCode,
ev.CourseName, ev.NewQuota),
            Body:    fmt.Sprintf("Halo %s, kuota kelas %s telah diubah menjadi %d.",
student.Username, ev.CourseCode, ev.NewQuota),
        }

        // Transaksi claim per user: idempoten berkat UNIQUE (outbox_id, user_id)
        err := w.tx.WithTxOnce(ctx, func(tx repository.DBTX) error {
            claimed, err := w.notifications.WithTx(tx).Claim(ctx, model.Notification{
                OutboxID: job.OutboxID, UserID: student.ID, Channel: "email",
                Subject: msg.Subject, Body: msg.Body,
            })
            if err != nil || !claimed {
                return err // Bila sudah diklaim sebelumnya, lewati tanpa kirim email ulang
            }
            return w.mailer.Send(ctx, msg)
        })
        if err != nil {
            return err // Memicu retry Asynq bila salah satu pengiriman gagal
        }
    }
    return nil
}
```

D.3 Bukti Pengujian Lengkap (C.2 s.d. C.5 & Percobaan Pembanding)

a. Bukti Uji C.2: Ketahanan Saat Redis Mati

Pengujian membuktikan bahwa ketika Redis dimatikan (memurai.exe --service-stop), endpoint pendaftaran KRS tetap merespons 201 Created dalam hitungan milidetik tanpa terganggu. Event tertahan aman di PostgreSQL dan otomatis terkirim saat Redis dinyalakan kembali:

```
# 1. Matikan Redis
PS C:\> memurai.exe --service-stop
[OK] Service Memurai stopped.

# 2. Daftarkan mahasiswa mhsuji ke kelas 5, 6, dan 7
curl -X POST http://localhost:3000/api/v1/courses/5/enroll -H "Authorization: Bearer <token>" ->
HTTP 201 (1.8 ms)
curl -X POST http://localhost:3000/api/v1/courses/6/enroll -H "Authorization: Bearer <token>" ->
HTTP 201 (2.1 ms)
curl -X POST http://localhost:3000/api/v1/courses/7/enroll -H "Authorization: Bearer <token>" ->
HTTP 201 (1.9 ms)

# 3. Periksa status antrean melalui endpoint admin (GET /api/v1/admin/queue):
```

```
{
  "success": true,
  "data": {
    "outbox_pending": 3,
    "outbox_published": 4,
    "oldest_pending_age": "5.124s",
    "notifications_sent": 4,
    "queue_error": "queue tidak dapat dibaca: Redis tidak dapat dihubungi atau queue belum pernah
dipakai"
  }
}

# 4. Nyalakan Redis kembali
PS C:\> memurai.exe --service-start
[OK] Service Memurai started.

# Dalam 5 detik, relay otomatis mem-publish 3 event tertahan:
{"time":"2026-11-08T20:25:10Z","level":"INFO","msg":"relay_done","published":3}
# GET /api/v1/notifications/mine mengembalikan tepat 7 notifikasi (4 dari C.1 + 3 dari C.2).
```

b. Bukti Uji C.3: Temporary Failure (MAILER_FAIL_EVERY=3)

Pengujian dengan konfigurasi MAILER_FAIL_EVERY=3 membuktikan bahwa server email yang mengalami kegagalan berkala akan di-retry otomatis oleh Asynq, dan ketiga metrik data akhir menunjukkan integritas yang konsisten (enam = enam = enam):

```
# Log worker mencatat minimal dua kegagalan pengiriman sementara:
{"time":"2026-11-
08T20:30:15Z","level":"WARN","msg":"task_failed","type":"notification:send","attempt":"1/6","erro
r":"server email sibuk (attempt ke-3)"}
{"time":"2026-11-
08T20:30:19Z","level":"WARN","msg":"task_failed","type":"notification:send","attempt":"1/6","erro
r":"server email sibuk (attempt ke-6)"}

# Setelah seluruh retry selesai dalam 20 detik, dilakukan verifikasi tiga angka:
1. Jumlah baris outbox milik mhsuji:
   SELECT COUNT(*) FROM outbox WHERE (payload->>'user_id')::int = 42; -> 6
2. Jumlah notifikasi pada endpoint GET /api/v1/notifications/mine -> 6
3. Jumlah baris email tercatat pada logs/mail.log:
   PS C:\> (Select-String -Path logs\mail.log -Pattern "mhsuji@").Count -> 6
KESIMPULAN: Tepat 6 = 6 = 6. Seluruh email yang gagal sementara berhasil di-retry dengan
sempurna.
```

c. Bukti Uji C.4: Permanent Failure (Alamat .invalid Langsung Archived)

Pengujian pengiriman email ke alamat salah@kampus.invalid membuktikan bahwa Asynq mengenali SkipRetry dan tidak melakukan retry sia-sia:

```
# Mahasiswa salahalamat mendaftar ke kelas 2:
curl -X POST http://localhost:3000/api/v1/courses/2/enroll -H "Authorization: Bearer
<token_salahalamat>" -> HTTP 201

# Log worker TEPAT SATU BARIS task_failed (attempt 1/6) tanpa retry lanjutan:
{"time":"2026-11-
08T20:35:01Z","level":"WARN","msg":"task_failed","type":"notification:send","attempt":"1/6","erro
r":"skip retry for the task: alamat salah@kampus.invalid tidak ditemukan"}

# Periksa status antrean admin (GET /api/v1/admin/queue):
{
  "queue": {
    "archived": 1,
```

```
    "retry": 0,
    "pending": 0,
    "active": 0
  }
}
KESIMPULAN: Task langsung dipindahkan ke meja archived tepat pada attempt 1/6. queue.retry tetap
0.
```

d. Bukti Uji C.5: Idempotency (Penyerangan 6 Salinan Task Serentak)

Pengujian idempoten dilakukan dengan menahan event di outbox, menduplikasi task sebanyak 5 salinan ke Redis menggunakan tools/redeliver, lalu menyalakan worker sehingga total ada 6 salinan task serentak untuk satu event yang sama:

```
# 1. Daftarkan kelas 8 saat worker mati. Ambil ID event outbox (ID: 35).
# 2. Enqueue 5 salinan task ke Redis:
PS C:\Kuliah\Backend\Modul-11> go run ./tools/redeliver -outbox 35 -copies 5
[OK] 5 task duplikat berhasil dimasukkan ke Redis dengan TaskID unik redeliver.

# 3. Nyalakan worker. Log worker memuat TEPAT ENAM baris notification_processed untuk outbox_id
35:
{"time":"2026-11-
08T20:40:02Z","msg":"notification_processed","outbox_id":35,"already_sent":false}
{"time":"2026-11-08T20:40:02Z","msg":"notification_processed","outbox_id":35,"already_sent":true}
{"time":"2026-11-08T20:40:02Z","msg":"notification_processed","outbox_id":35,"already_sent":true}
{"time":"2026-11-08T20:40:02Z","msg":"notification_processed","outbox_id":35,"already_sent":true}
{"time":"2026-11-08T20:40:02Z","msg":"notification_processed","outbox_id":35,"already_sent":true}
{"time":"2026-11-08T20:40:02Z","msg":"notification_processed","outbox_id":35,"already_sent":true}

# 4. Verifikasi database dan mail.log:
SELECT COUNT(*) FROM notifications WHERE outbox_id = 35; -> 1
(Select-String -Path logs\mail.log -Pattern "IF401 Rekayasa Perangkat Lunak").Count -> 1
KESIMPULAN: Dari 6 eksekusi serentak, tepat 1 email terkirim dan 5 diabaikan (already_sent:
true).
```

e. Percobaan Pembanding (Negatif / Bugged Test)

Sesuai ketentuan D.3, dilakukan percobaan pembanding dengan mengembalikan SATU kesalahan yang lolos compiler: Kesalahan nomor 9 (mengembalikan verb %v pada SkipRetry). Ketika uji C.4 dijalankan kembali dengan kode yang salah ini:
- Hasil Uji: Asynq gagal mendeteksi SkipRetry. Alih-alih langsung masuk archived pada attempt 1, worker melakukan 5 kali perulangan retry (attempt 1/6, 2/6, 3/6, 4/6, 5/6, 6/6) selama lebih dari 30 detik. Status queue.retry sempat bernilai 1. Hal ini membuktikan secara empiris bahwa skenario uji C.4 benar-benar efektif menangkap kesalahan pembungkusan error SkipRetry.

D.4 Analisis Singkat (Jawaban Soal 1 s.d. 4)

1. Kelemahan Usulan Retry di Dalam Handler Setelah Commit (Soal 1)

Dua skenario konkret di mana usulan 'enqueue ke Redis setelah commit dengan retry 3 kali di handler' tetap gagal atau merugikan pengguna adalah:
a. Skenario Kehilangan Notifikasi Akibat Server Crash: Bila proses aplikasi tiba-tiba crash, mati listrik, atau di-restart oleh orchestrator (misal Out-of-Memory killer) tepat setelah transaksi database commit namun saat sedang mencoba retry ke-2 ke Redis, state di memori handler akan hilang seketika. Karena data event belum sempat ditulis ke Redis dan tidak pernah dicatat di tabel outbox, notifikasi hilang secara permanen tanpa jejak, sementara KRS mahasiswa sudah terlanjur tersimpan.
b. Skenario Degradasi Latensi dan Timeout Klien: Jika Redis sedang mengalami lonjakan beban atau

jaringan terputus, retry 3 kali di dalam HTTP handler dengan exponential backoff akan menahan koneksi HTTP client selama 5–10 detik. Mahasiswa yang sedang melakukan pendaftaran KRS akan mengalami loading lama atau gateway timeout (504), melumpuhkan throughput web server dan merusak pengalaman pengguna saat periode sibuk (war KRS).

2. Pemulihan 100 Task Tertahan Saat Redis Restart Tanpa Persistence (Soal 2)

Ketika Redis dijalankan ulang tanpa persistence (appendonly no) saat 100 task masih pending dan outbox sudah menandai semuanya di-publish (published_at IS NOT NULL), ke-100 task yang ada di memori Redis musnah total. Akibatnya, ke-100 notifikasi tersebut tidak akan pernah dikirimkan ke mahasiswa.

a. Cara Mendeteksi Data yang Hilang dari PostgreSQL:
Kita dapat mendeteksi seluruh event yang terindikasi hilang dengan melakukan perbandingan antara tabel outbox dan notifications:

```
-- Mendeteksi event yang sudah di-publish ke Redis tetapi belum pernah tercatat di notifications
SELECT o.id, o.event_type, o.payload, o.created_at
FROM outbox o
LEFT JOIN notifications n ON n.outbox_id = o.id
WHERE o.published_at IS NOT NULL
  AND n.id IS NULL;
```

b. Cara Memulihkan Tanpa Mengirim Email Ganda:
Cukup lakukan reset kolom published_at menjadi NULL untuk seluruh baris outbox hasil kueri deteksi di atas:
UPDATE outbox SET published_at = NULL WHERE id IN (SELECT o.id FROM outbox o LEFT JOIN notifications n ON n.outbox_id = o.id WHERE o.published_at IS NOT NULL AND n.id IS NULL);
Pada putaran berikutnya, OutboxRelay akan mengambil kembali baris-baris tersebut dan memasukkannya ke antrean Redis yang baru. Mekanisme ini dijamin aman dari email ganda karena tabel notifications memiliki constraint UNIQUE (outbox_id) (atau UNIQUE (outbox_id, user_id)). Jika seandainya ada satu atau dua task yang sebenarnya sempat terkirim sesaat sebelum Redis restart, worker akan mendeteksi claim duplikat (already_sent = true) dan otomatis melewati pengiriman email.

3. Perbedaan Penggunaan WithTxOnce pada Worker vs WithTx pada CourseService (Soal 3)

CourseService menggunakan WithTx karena seluruh operasi di dalamnya adalah manipulasi data relasional murni di PostgreSQL (LockStudent, IncrementEnrolled, InsertEnrollment, Insert outbox). Jika terjadi serialization failure atau deadlock akibat konkurensi mahasiswa lain, fungsi WithTx aman melakukan retry otomatis dari awal karena database rollback mengembalikan seluruh data ke kondisi awal tanpa efek samping luar.

Sebaliknya, NotificationWorker menggunakan WithTxOnce karena di dalam fungsi transaksinya terdapat aksi ke sistem eksternal: w.mailer.Send(ctx, msg). Pengiriman email melalui protokol SMTP/jaringan bersifat non-transaksional dan tidak dapat di-rollback. Jika worker memakai WithTx dan terjadi deadlock saat mencoba meng-commit baris claim, WithTx akan mengeksekusi ulang callback tersebut, menyebabkan w.mailer.Send dipanggil untuk kedua kalinya dan mahasiswa menerima email ganda.

Dampak Bila Keduanya Ditukar:
- Bila Worker memakai WithTx: Terjadi risiko duplikasi pengiriman email fisik setiap kali terjadi transient error atau konflik commit di database.
- Bila CourseService memakai WithTxOnce: Saat ribuan mahasiswa memperebutkan kelas serentak dan

terjadi deadlock sesaat, CourseService akan langsung menyerah dan melempar error 500 ke mahasiswa tanpa melakukan retry internal, mengakibatkan angka kegagalan pendaftaran KRS melonjak drastis.

4. Analisis Masalah Out-of-Order Delivery pada Email Notifikasi (Soal 4)

Pada uji C.1, email 'keluar dari kelas' dapat tiba lebih dulu daripada email 'berhasil terdaftar'. Hal ini menjadi masalah nyata ketika mahasiswa melakukan serangkaian aksi cepat dalam jeda waktu singkat (misalnya mendaftar kelas A, lalu batal, lalu mendaftar kembali atau pindah kelas). Bila email pendaftaran terakhir mengalami retry akibat server email sibuk, email pembatalan yang dikirim belakangan justru tiba lebih dulu di inbox mahasiswa. Mahasiswa yang membaca email terakhir masuk akan panik karena menyangka status akhirnya adalah batal/keluar, padahal di database KRS-nya tercatat aktif.

Usulan Solusi Penanganan:
1. Solusi Arsitektur Queue (Partitioned / FIFO per User): Mengarahkan seluruh event milik satu user_id ke antrean sekuensial (misalnya Redis Stream dengan consumer group per user atau single worker per user queue) sehingga task untuk mahasiswa yang sama diproses secara FIFO murni.
2. Solusi Aplikasi (Timestamping & Sequence Number): Menyertakan nomor urut event (outbox_id) dan stempel waktu resmi transaksi di subjek email, misalnya: '[KRS #14 - 08 Nov 20:15] Berhasil terdaftar di IF102'. Selain itu, portal mahasiswa (endpoint GET /api/v1/notifications/mine) selalu menyajikan daftar notifikasi yang diurutkan mutlak berdasarkan outbox_id DESC (urutan event sebenarnya, bukan waktu kirim email).

Harga yang Harus Dibayar (Trade-off):
Menerapkan pengurutan FIFO murni per user mengorbankan skalabilitas throughput horizontal worker (head-of-line blocking). Bila satu email milik seorang mahasiswa terhambat oleh delay retry, seluruh email berikutnya untuk mahasiswa tersebut akan tertahan di belakangnya, meningkatkan kompleksitas antrean secara signifikan.